

Assignment 5

Design Under Test

The DUT is router.v, provided separately by the instructor. Read and understand its ports, parameters, and expected protocol/timing before you start writing any testbench code — your driver and monitor timing must match the DUT's interface behavior exactly.

Required Environment Structure

Build a full UVM testbench with the following components:

- Top-level env class instantiating all sub-components.
- One agent only containing a driver, a monitor, and a sequencer with its active/passive behavior controlled through `uvm_active_passive_enum`. Use the config object to switch the agent between `UVM_ACTIVE` and `UVM_PASSIVE`.
- Driver: drives sequence items onto the DUT interface following the correct protocol timing.
- Monitor: samples interface activity and broadcasts observed transactions.
- Sequencer and at least one representative sequence generating meaningful stimulus.
- Sequence item(s) modeling router transactions.
- Build a Coverage to collect and sample covergroups and Scoreboard to self-verify the design, No connections needed.

Config Object and config_db

- Define at least one `uvm_object`-derived configuration class (e.g. `router_config`) holding environment-wide settings (number of ports, active/passive mode, coverage enable, etc.).
- Use `uvm_config_db#(...)::set()` from the test (or higher level) to configure the environment.
- Use `uvm_config_db#(...)::get()` inside `build_phase` of the relevant components to retrieve that configuration.
- Show the agent's `is_active` field being controlled through the config object rather than hardcoded.

Reporting Requirement: Verbosity Demonstration

This is the core graded exercise of the assignment. It proves you understand how UVM verbosity filtering actually works, not just how to instantiate components.

1. Build phase messages: inside the `build_phase` of every single component in the environment (env, agents, driver, monitor, sequencer, scoreboard, coverage collector), print an identifying message using `uvm_info`. Repeat this print four times, once at each verbosity level: `UVM_LOW`, `UVM_MEDIUM`, `UVM_HIGH`, and `UVM_FULL`.
2. Four simulation runs: run the full testbench four separate times, changing only the command-line verbosity plusarg each time: `+UVM_VERBOSITY=UVM_LOW`, then `UVM_MEDIUM`, then `UVM_HIGH`, then `UVM_FULL`.

Automation

Provide a single `run.do` that compiles the design and testbench and can be re-invoked for each of the four verbosity runs by only changing the `+UVM_VERBOSITY` plusarg. Do not hand-edit source files between runs.

Deliverables

Submit two files:

- A zip/rar archive containing: all implemented .sv files, the router.v design file corrected from bugs if found, and the run.do file used to automate all four runs.
- A separate PDF report containing: code snippets for every implemented UVM file, log snippets for all four runs showing correct construction of every component/object and correctly filtered print messages, and a snippet of the .do file used.

Name your files exactly as follows :

- Archive: `YourName_Assignment_5.rar`: (e.g. `Hassan_Khaled_Assignment_5.rar`)
- Report: `YourName_Assignment_5.pdf`: (e.g. `Hassan_Khaled_Assignment_5.pdf`)